

Start from what already works.

You list the channels in your niche. Eight small steps later you know which of their breakout topics are repeatable, and which are also rising on Google. This guide walks through every step with a real example.

Success leaves clues. Most of them are noise.

Every niche has videos that blew up. Copying them rarely works, because a video's views have four causes and only one of them is the topic.

$$\text{views} \approx \text{channel} \times \text{topic} \times \text{packaging} \times \text{luck}$$

This pipeline starts from the breakout videos of the channels you list, and then does the work of separating the topic from everything else. A real result, from 24 AI and tech channels:

TOPIC	REPEATABILITY	CHANNELS	SEARCHES / MONTH	VERDICT
GPT-6 Astra	4.57	13	1,000	make it
Claude Code	4.52	18	2,900	make it
PDF compression	2.61	12	40,500	make it
Gemma 4	2.33	8	22,200	make it
AI agents	3.54	16	1,600	repeatable, but search is flat
AI login automation	1.40	10	5,400	rising, but not repeatable

Six of the eight topics of the run. Searches are for the topic's most searched Google keyword.

THE PRINCIPLE BEHIND IT

One viral video proves little. The same topic working for 18 channels proves a lot.

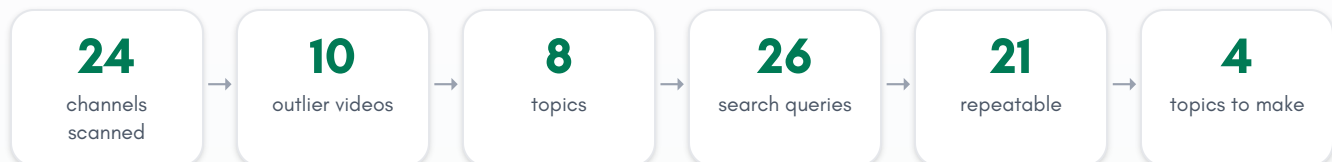
Packaging and luck differ from channel to channel. When many unrelated channels all beat their own normal with the same topic, the thing they share is the topic.

Eight steps. Each one asks one question.

The first four steps happen on YouTube and answer "is it repeatable?". The last four happen on Google and answer "is it rising?".

- | | | | |
|---|----------------|---------------------------|---|
| 1 | YouTube | Find outliers | Which videos far outperformed their own channel? |
| 2 | LLM | Outliers to topics | Which topics are those videos about? |
| 3 | YouTube | Search queries | What do viewers really type to find them? (a: YouTube, b: LLM) |
| 4 | YouTube | Repeatability | Has each search query worked for many channels? |
| 5 | LLM | Google seeds | How would people google each topic? (a: YouTube phrasings, b: LLM) |
| 6 | Google | Keyword ideas | What do people search for around each topic? (a: Google, b: keep the biggest) |
| 7 | LLM | Pick keywords | Which of those keywords are really about the topic? |
| 8 | Rule | Scoreboard | Which topics are repeatable and rising? |

The same run, as a funnel



Real numbers from the run on the previous page. Steps 5 to 7 then looked at 5,876 Google keywords to find the 50 that are really about the eight topics.

FOUR KINDS OF STEP

YouTube and **Google** steps fetch facts. **Rule** steps are plain arithmetic. **LLM** steps use an AI model only to translate between how people talk: a video title, a YouTube search, a Google search. The numbers never come from the model.

Which videos far outperformed their own channel?

For each channel on your list, the pipeline finds the channel's normal: the median views of its last 50 uploads. A video at 5 times that normal is an outlier. The 10 biggest go on.

GOES IN

@nicksaraev

@mreflow

@howiaipodcast

@PavanLalwani

and **20** more channels from config.py.

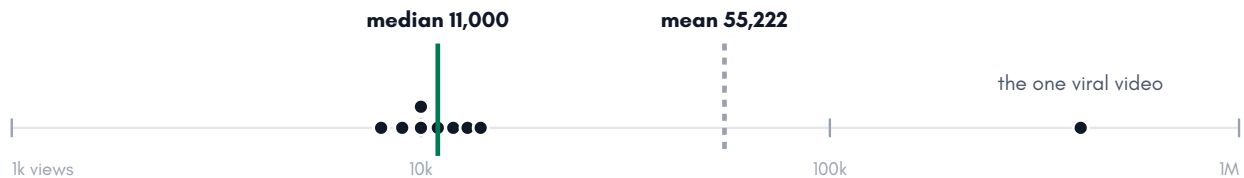
COMES OUT

"Convert Image PDF to Editable Text" **6.1M** vs **28,012** normal = **217x**

"CLAUDE CODE FULL COURSE 4 HOURS" **2.5M** vs **79,758** = **31.5x**

"Build Your First AI Agent in 10 Minutes" **315,881** vs **14,753** = **21.4x**

10 outlier videos.



Nine uploads of one imaginary channel. The average says a normal video gets 54,000 views. The median says 11,000, which is the truth.

WHY IT WORKS

Compare a channel with itself.

Raw views mostly measure how big the channel is. A channel's own normal cancels that out. The normal is the median, not the average, because one viral video drags an average far above what the channel usually gets.

WHERE IT FALLS SHORT

- It can only find topics the channels on your list have made. The list is the most important setting.
- One outlier is one data point. It may be luck or a great thumbnail. That is why steps 3 and 4 exist.
- Older videos have had more time to collect views.

TRY IT `uv run python -m steps.find_outliers @Fireship`

SAVED AS `01_find_outliers.json`

Which topics are those videos about?

An AI model reads each outlier's title and the first lines of its description, and names the topic underneath, with the plain search query a viewer would type. Videos about the same thing become one topic.

GOES IN

"GPT-6 Astra Is Finally Here (And It's REALLY Good)"

description: Here's my first look at GPT-6-Astra after spending some time using it myself ...

"I Tested GPT 6 Astra vs Fable 5.1 (No Hype ...)"

"CLAUDE CODE FULL COURSE 4 HOURS: Build & Sell"

and 7 more outlier videos, each with its description.



COMES OUT

GPT-6 Astra · 2 videos "gpt 6 astra"

Claude Code · 1 "claude code
video tutorial"

PDF compression "compress pdf without
· 1 video losing quality"

8 topics, each with a search query.

WHY IT WORKS

A title is packaging. The topic is what is underneath.

"Is Finally Here (And It's REALLY Good)" is how one creator sold the video. The description is where creators usually say plainly what a video covers. Reading through the packaging is a language task, so it goes to a language model.

WHERE IT FALLS SHORT

- Many descriptions open with sponsor links and say nothing about the video. Then the title is all the model has.
- The search query is the model's guess. It can be too specific for what people really type. Step 3 corrects for that.
- A different model may split or merge topics differently.

TRY IT `uv run python -m steps.outliers_to_topics` (one small LLM call)

SAVED AS `02_outliers_to_topics.json`

What does YouTube's search box suggest?

Type something into YouTube and it finishes your sentence. Those suggestions are what viewers really type, most popular first. The pipeline asks for them, for every topic's query. It is free and needs no key.

GOES IN

claude code tutorial

One topic's search query.



COMES OUT

claude code tutorial for beginners

claude code tutorial mac

claude code tutorial vscode

claude code tutorial for developers

claude code tutorial for non coders

and 4 more suggestions.

WHY IT WORKS

Ask the crowd how they phrase it.

A model guessed the query in step 2.

YouTube's search box knows what millions of viewers actually typed. Real phrasings beat guessed ones, because the search results, and so the evidence, depend on the exact words.

WHERE IT FALLS SHORT

- It gives an order, not numbers. You learn what is popular, not how popular.
- Long, specific queries get few suggestions or none. Short queries work best.
- It is an unofficial address that YouTube could change without notice. If it fails, the pipeline goes on with the topic's own query.

TRY IT `uv run python -m steps.youtube_suggestions "claude code"`

SAVED AS `03_a_youtube_suggestions.json`

Which phrasings are worth a search of their own?

An AI model picks up to 3 of YouTube's suggestions per topic: same language, still on topic, and a different angle or audience. Each pick is scored in the next step as an idea of its own.

GOES IN

claude code	+ 9
tutorial	suggestions
gpt 6 astra	+ 9 suggestions
ai agents	+ 9 suggestions

8 topics with their suggestions.



COMES OUT

claude code tutorial	the topic's own query
claude code tutorial for beginners	picked
claude code tutorial vscode	picked
claude code tutorial for non coders	picked

26 search queries for the 8 topics.

WHY IT WORKS

The same topic can win under one phrasing and lose under another.

"For beginners" and "for non coders" are different audiences, with different videos competing. In this run the first scored 4.52 and the second 2.56. The guardrail: a pick that is not in YouTube's list is thrown away, so the model cannot invent a query.

WHERE IT FALLS SHORT

- Every extra query costs a YouTube search: 26 searches are 2,600 of your 10,000 daily units.
- "A different angle" is a judgement. Expect an odd pick now and then, such as "gpt 6 astra roblox".

TRY IT `uv run python -m steps.query_variants "claude code"`

SAVED AS `03_b_query_variants.json`

Has each search query worked for many channels?

One YouTube search per query. Every result is compared with what **its own channel** normally gets. A video at 4 times its channel's normal is a hit. Every hit adds to the query's score, and newer hits, and hits from channels near your size, add more.

GOES IN

claude code tutorial for beginners

gpt 6 astra free

ai agents tutorial

python ocr pdf to text

and **22** more search queries. One YouTube search each.



COMES OUT

gpt 6 astra free 15 hits · 13 channels · **4.57**

claude code tutorial for beginners 25 hits · 18 channels · **4.52**

ai agents tutorial 25 hits · 16 channels · **3.54**

python ocr pdf to text 8 hits · 6 channels · **0.66**

The last number on each line is the query's score.

WHAT "REPEATABLE" MEANS

A query is repeatable when it passes two tests at once: its hits come from **6 or more different channels**, and its **score is 2.0 or more**. A score of 1.0 is worth one brand-new hit from a channel exactly your size. **21** of the 26 queries pass. "python ocr pdf to text" has just enough channels, but its score of **0.66** is far under the bar. The next page works through one score by hand.

WHY IT WORKS

Divide by the channel, then demand many channels.

A video's views come from the channel, the topic, the packaging and luck. Dividing by what the channel normally gets cancels the channel. Asking for hits on many unrelated channels cancels packaging and luck, because those differ from channel to channel. What is left is the topic.

WHERE IT FALLS SHORT

- It only sees the videos YouTube's search returns. Most flops on the topic stay invisible.
- The bars (4 times normal, 6 channels, the score) are judgement calls. You can change them.
- It measures the topic, not you. A repeatable topic still needs a good title, thumbnail and video.

TRY IT `uv run python -m steps.score_repeatability "claude code"`

SAVED AS 04_score_repeatability.json

How "claude code tutorial for beginners" got its score of 4.52.

1

Search, and set aside what can't be compared

YouTube returns up to 50 videos for the query. Videos older than 2 years, or with under 1,000 views, are set aside.

2

Is each video a hit? $\text{views} \div \text{its channel's normal} \geq 4$

VIDEO	VIEWS	CHANNEL'S NORMAL	RESULT
"FULL Claude Code Tutorial ..." · Productive Dude	14,139	799	17.7x hit
"The Ultimate Beginner's Guide ..." · Metics Media	925,616	14,761	62.7x hit
"I Have Spent 1000+ Hours ..." · The Coding Sloth	640,550	147,815	4.3x hit

This search found 25 hits, on 18 different channels. A video at 2 times normal adds nothing.

3

How much does each hit count? $\text{freshness} \times \text{closeness}$

Freshness halves every 180 days: today 1.0, six months 0.5, a year 0.25. **Closeness** is 1.0 for a channel your size, 0.5 for one 10 times bigger or smaller, 0.33 for 100 times, 0.25 for 1,000 times. In this run your channel's normal is about 140 views.

HIT	AGE → FRESHNESS	CHANNEL SIZE → CLOSNESS	COUNTS FOR
Productive Dude	157 days → 0.55	5.6 times yours → 0.57	$0.55 \times 0.57 = 0.31$
Metics Media	42 days → 0.85	104 times yours → 0.33	$0.85 \times 0.33 = 0.28$
The Coding Sloth	34 days → 0.88	1,037 times yours → 0.25	$0.88 \times 0.25 = 0.22$

The 62.7x hit counts for less than the 17.7x hit. How big a hit was does not matter. How recent it is, and how much the channel is like yours, does.

4

Add up all 25 hits

$$0.31 + 0.31 + 0.28 + 0.26 + 21 \text{ smaller ones} = 4.52$$

5

The verdict: two tests, both must pass

SEARCH QUERY	CHANNELS WITH A HIT (6 OR MORE)	SCORE (2.0 OR MORE)	VERDICT
claude code tutorial for beginners	18 pass	4.52 pass	repeatable
python ocr pdf to text	6 pass	0.66 fail	not repeatable

6

A topic is judged by its best phrasing

The topic Claude Code was searched four ways: "for beginners" 4.52, "claude code tutorial" 3.37, "vscode" 3.12, "for non coders" 2.56. The topic takes the best one, 4.52, and that phrasing is the one to build your title around.

How would people google each topic?

Step 6 will ask Google how many people search for each topic. Google's tool needs short **seed keywords**, and people phrase things differently there than on YouTube. So this step translates: (a) each topic gets back its phrasings from step 4, then (b) an AI model writes three Google seeds for it.

GOES IN

Claude Code, with the 4 phrasings scored in step 4:

claude code tutorial for beginners
claude code tutorial
claude code tutorial vscode
claude code tutorial for non coders



COMES OUT

Claude Code, with 3 seed keywords for Google:

claude code
claude code tutorial
claude code vscode

Step 6 sends exactly these three to Google.

TWO MORE OF THE 8 TOPICS. 8 TOPICS × 3 SEEDS = 24 SEED KEYWORDS.

TOPIC	GOES IN: YOUTUBE PHRASINGS	COMES OUT: GOOGLE SEEDS	WHAT HAPPENED
PDF compression	compress pdf without losing quality · compress pdf to 100kb without losing quality · 2 more	compress pdf reduce pdf size pdf compressor offline	Shortened, and reworded the way people google it.
AI login automation	ai login automation	login automation ai automation automated login	Drifted. The video was about AI that logs in for you. These seeds are about logins in general.

WHY IT WORKS

Translate before you ask.

On YouTube people type "compress pdf without losing quality". Google's keyword tool works best with short seeds such as "compress pdf". Asking Google in YouTube's language would return little.

WHERE IT FALLS SHORT

- Everything after this depends on the seeds. The last row of the table drifted, and step 7 shows where that leads.
- It is a model's judgement, so check the saved file when a verdict looks odd.

TRY IT `uv run python -m steps.google_seeds` (one small LLM call) SAVED AS `05_a_...` and `05_b_google_seeds.json`

What do people search for around each topic?

For each topic, the pipeline asks Google one question: what do people type around these three seeds, and how often? (a) Google sends back a list of keywords with their searches per month. (b) A simple rule then keeps only the keywords big enough to matter: 200 or more searches a month, and at most the 40 biggest.

GOES IN

The 3 seeds of **Claude Code**:

claude code

claude code tutorial

claude code vscode

One request to Google, for this topic alone.



COMES OUT

Everything Google sent back, with searches a month:

claude code	550,000	kept
codegeass	135,000	kept
claude code tutorial	2,900	kept
claude code vscode	1,600	kept
lelouch movie	70	dropped
claudia ide	10	dropped

6 keywords came back. 2 are under 200 searches, so 4 go on. "codegeass" is an anime: Google matches words, not meaning. Step 7 removes it.

WHY IT WORKS

One request per topic, so no topic is crowded out.

Google blends the seeds of one request into one list. With all topics in one request, PDF keywords, with their huge volumes, would bury Claude Code. Asking separately gives every topic its own list.

WHERE IT FALLS SHORT

- Most of what Google sends back is not about the topic. The next step sorts that out.
- A brand-new topic has few keywords. GPT-6 Astra got 4, so its verdict rests on thin data.
- Keeping the biggest favours generic keywords over precise ones.

TRY IT `uv run python -m steps.keyword_ideas "claude code"`

SAVED AS 06_a_... and 06_b_...json

From 5,876 keywords down to 173.

Claude Code was a small example: Google sent back only 6 keywords. For a busy topic it sends back thousands, and most are tiny. Here is the same rule on a big topic.

PDF COMPRESSION. THE SEEDS WERE "COMPRESS PDF", "REDUCE PDF SIZE" AND "PDF COMPRESSOR OFFLINE".

2,086 keywords came back from Google

- 1,832 Too small. Under 200 searches a month.

254 left
- 214 Not among the 40 biggest. The list is sorted by searches and cut at 40.

40 left
- =

40 Go on to step 7, each with 48 months of searches and its trend.

THE SAME TWO PARTS FOR ALL 8 TOPICS: (A) WHAT GOOGLE SENT BACK, (B) WHAT THE RULE KEPT

TOPIC	A. GOOGLE SENT BACK	OF THOSE, 200+ SEARCHES A MONTH	B. KEPT (AT MOST 40)
PDF OCR	3,057	443	40
PDF compression	2,086	254	40
AI login automation	420	42	40
AI opportunities	153	28	28
AI agents	142	14	14
Gemma 4	8	4	4
Claude Code	6	4	4
GPT-6 Astra	4	3	3
All 8 topics	5,876	792	173

WHY CUT AT 40?

The next step is an AI model reading the list. Short lists get read carefully.

A keyword with 20 searches a month cannot tell you whether a topic is rising: at that size the numbers are noise. And a model asked to judge 2,000 keywords at once gets sloppy and costs more. So the rule hands it the few that matter. Both numbers, 200 and 40, are settings in config.py.

Which of those keywords are really about the topic?

An AI model reads each topic's candidate keywords and keeps only the ones a viewer of that video would search for. The topic's search trend is then read from the most searched keyword it kept.

GOES IN

Claude Code	4 candidates
AI agents	14 candidates
AI login automation	40 candidates

173 candidate keywords for the 8 topics.

COMES OUT

Claude Code	claude code tutorial · claud code vscode
AI agents	ai agents examples · ai agent tutorial · 3 more
AI login automation	power automate login

50 keywords picked.

WHY IT WORKS

Judge a topic only by keywords that are truly about it.

If "club automation login", a real suggestion from Google, counted for the topic "AI login automation", its search trend would describe something else entirely. The same guardrail as before applies: the model may only pick from Google's list.

WHERE IT FALLS SHORT

- Here the pipeline visibly falls short. "Power automate login" is a Microsoft product, not AI that logs in for you. The seeds drifted in step 5 and no good keyword was on the list. So that topic's "rising" is not to be trusted.
- Always read the keyword next to a verdict.

TRY IT `uv run python -m steps.pick_topic_keywords` (one small LLM call) SAVED AS `07_pick_topic_keywords.json`

Which topics are repeatable and rising?

The last step puts the two tests side by side, one row per topic: the best YouTube phrasing and its score, the most searched Google keyword and its trend, and the verdict.

GOES IN

The topics, their YouTube scores and their Google keywords.



COMES OUT

Claude Code

make it

Gemma 4

make it

AI agents repeatable, but search is flat

AI opportunities repeatable, but search is shrinking

AI login automation rising, but not repeatable

4 topics to make, out of 8.

WHY IT WORKS

Every verdict carries its reason.

"Repeatable, but search is flat" is useful: the topic works on YouTube today, but the audience is not growing, so expect more competition for the same viewers. Because every step saved its work, you can walk back to the number behind any verdict.

WHERE IT FALLS SHORT

- "Make it" means the odds are good. It is not a promise. Your title, thumbnail and video still decide the result.
- The rising test rests on one keyword per topic. If that keyword is a poor match, the verdict is too.
- The data moves. Run it again before you commit weeks to a topic.

TRY IT `uv run python -m steps.scoreboard` (needs no keys)

SAVED AS 08_scoreboard.json and scoreboard.csv

What this pipeline can't tell you.

It turns other channels' results into evidence about topics. It is not an oracle. Knowing where it is weak is what lets you trust it where it is strong.

LIMIT

WHAT TO DO ABOUT IT

It only sees the channels you list

A topic none of them has made stays invisible. List the channels of your niche, including small ones, and refresh the list now and then.

You only see the winners

YouTube shows the videos that did well. The many that flopped on the same topic are mostly hidden. Asking for hits on 6 or more channels reduces this. It does not remove it.

Three steps are a model's judgement

Steps 2, 5 and 7 translate language, and translation can drift, as "AI login automation" showed. Guardrails keep the model from inventing data. They cannot make it always right.

It costs quota

Every search query costs 100 of your 10,000 daily YouTube units. The 26 searches of this run cost 2,600. Answers are kept for 24 hours, so running again the same day is free.

The thresholds are opinions

5 times normal, 4 times normal, 6 channels, 180 days. They are all named settings in config.py. Change one, run again, and see which verdicts move.

FIRST PRINCIPLE**A model you can't inspect, you can't improve.**

Every step saves a file with a summary, what went in and what came out. When a verdict surprises you, open the run folder and walk back, file by file, to the step that caused it.

When to use pipeline 1 instead

Use this pipeline to learn what is working in your niche right now, including topics you had no words for. Use pipeline 1, from keywords, when you already know what you want to cover and want to check it.

RUN IT YOURSELF

One command.

Set it up once and list the channels of your niche. A page opens in your browser that shows every step of the run.

```
cp .env.example .env          your YouTube and LLM keys
cp config.py.example config.py your channel, and the channels to scan
uv run pipeline2_from_youtube.py run it
```

Without Google Ads access the run stops after step 4, with the YouTube half of the answer. The README walks through the rest.